

程式設計 Function

張賢宗

長庚大學人工智慧學系

本投影片修改自書商、林仲志教授之原始版本

Learning Objectives

❖ Predefined Functions

- Those that return a value and those that don't

❖ Programmer-defined Functions

- Defining, Declaring, Calling
- Recursive Functions

❖ Scope Rules

- Local variables
- Global constants and global variables
- Blocks, nested scopes

Introduction to Functions

- ❖ Building Blocks of Programs
- ❖ Other terminology in other languages:
 - Procedures, subprograms, methods
 - In C++: functions
- ❖ I-P-O
 - Input – Process – Output
 - Basic subparts to any program
 - Use functions for these "pieces"

Predefined Functions

- ❖ Libraries full of functions for our use!
- ❖ Two types:
 - Those that return a value
 - Those that do not (void)
- ❖ Must "#include" appropriate library
 - e.g.,
 - <cmath>, <cstdlib> (Original "C" libraries)
 - <iostream> (for cout, cin)



widelab
Web Information and Data Engineering Laboratory

Using Predefined Functions

- Math functions very plentiful
 - Found in library `<cmath.h>`
 - Most return a value (the "answer")
- Example: `theRoot = sqrt(9.0);`
 - Components:
 - `sqrt` = name of library function
 - `theRoot` = variable used to assign "answer" to
 - `9.0` = argument or "starting input" for function
 - In I-P-O:
 - I = 9.0
 - P = "compute the square root"
 - O = 3, which is returned & assigned to theRoot

The Function Call

❖ Back to this assignment:

```
theRoot = sqrt(9.0);
```

- The expression "sqrt(9.0)" is known as a function *call*, or function *invocation*
- The argument in a function call (9.0) can be a literal, a variable, or an expression
- The call itself can be part of an expression:
 - `bonus = sqrt(sales)/10;`
 - A function call is allowed wherever it's legal to use an expression of the function's return type

A Larger Example:

Display 3.1 A Predefined Function That Returns a Value (1 of 2)

Display 3.1 A Predefined Function That Returns a Value

```
1  //Computes the size of a doghouse that can be purchased
2  //given the user's budget.
3  #include <iostream>
4  #include <cmath>
5  using namespace std;

6  int main( )
7  {
8      const double COST_PER_SQ_FT = 10.50;
9      double budget, area, lengthSide;

10     cout << "Enter the amount budgeted for your doghouse $";
11     cin >> budget;

12     area = budget/COST_PER_SQ_FT;
13     lengthSide = sqrt(area);
```

A Larger Example:

Display 3.1 A Predefined Function That Returns a Value (2 of 2)

```
14     cout.setf(ios::fixed);
15     cout.setf(ios::showpoint);
16     cout.precision(2);
17     cout << "For a price of $" << budget << endl
18         << "I can build you a luxurious square doghouse\n"
19         << "that is " << lengthSide
20         << " feet on each side.\n";

21     return 0;
22 }
```

SAMPLE DIALOGUE

Enter the amount budgeted for your doghouse \$25.00
For a price of \$25.00
I can build you a luxurious square doghouse
that is 1.54 feet on each side.

More Predefined Functions

❖ #include <cstdlib>

- Library contains functions like:
 - `abs()` // Returns absolute value of an int
 - `labs()` // Returns absolute value of a long int
 - `*fabs()` // Returns absolute value of a float
- `*fabs()` is actually in library <cmath>!
 - Can be confusing
 - Remember: libraries were added after C++ was "born," in incremental phases
 - Refer to appendices/manuals for details

More Math Functions

❖ `pow(x, y)`

- Returns x to the power y
double result, $x = 3.0$, $y = 2.0$;
`result = pow(x, y);`
`cout << result;`
 - Here 9.0 is displayed since $3.0^{2.0} = 9.0$

❖ Notice this function receives two arguments

- A function can have any number of arguments, of varying data types

Even More Math Functions:

Display 3.2 Some Predefined Functions (1 of 2)

Display 3.2 Some Predefined Functions

NAME	DESCRIPTION	TYPE OF ARGUMENTS	TYPE OF VALUE RETURNED	EXAMPLE	VALUE	LIBRARY HEADER
sqrt	Square root	double	double	sqrt(4.0)	2.0	cmath
pow	Powers	double	double	pow(2.0, 3.0)	8.0	cmath
abs	Absolute value for int	int	int	abs(-7) abs(7)	7 7	cstdlib
labs	Absolute value for long	long	long	labs(-70000) labs(70000)	70000 70000	cstdlib
fabs	Absolute value for double	double	double	fabs(-7.5) fabs(7.5)	7.5 7.5	cmath

Even More Math Functions:

Display 3.2 Some Predefined Functions (2 of 2)

ceil	Ceiling (round up)	double	double	ceil(3.2) ceil(3.9)	4.0 4.0	cmath
floor	Floor (round down)	double	double	floor(3.2) floor(3.9)	3.0 3.0	cmath
exit	End program	int	void	exit(1);	None	cstdlib
rand	Random number	None	int	rand()	Varies	cstdlib
srand	Set seed for rand	unsigned int	void	srand(42);	None	cstdlib

Predefined Void Functions

- ❖ No returned value
- ❖ Performs an action, but sends no "answer"
- ❖ When called, it's a statement itself
 - `exit(1);` // No return value, so not assigned
 - This call terminates program
 - void functions can still have arguments
- ❖ All aspects same as functions that "return a value"
 - They just don't return a value!



widelab
Web Information and Data Engineering Laboratory

Random Number Generator

- ❖ Return "randomly chosen" number
- ❖ Used for simulations, games
 - `rand()`
 - Takes no arguments
 - Returns value between 0 & RAND_MAX
 - Scaling
 - Squeezes random number into smaller range
 $\text{rand()} \% 6$
 - Returns random value between 0 & 5
 - Shifting
 $\text{rand()} \% 6 + 1$
 - Shifts range between 1 & 6 (e.g., die roll)



widelab
Web Information and Data Engineering Laboratory

Random Number Seed

❖ Pseudorandom numbers

- Calls to `rand()` produce given "sequence" of random numbers

❖ Use "seed" to alter sequence `srand(seed_value);`

- void function
- Receives one argument, the "seed"
- Can use any seed value, including system time:
`srand(time(0));`
- `time()` returns system time as numeric value
- Library `<time>` contains `time()` functions

Random Examples

- ❖ Random double between 0.0 & 1.0:
 $(\text{RAND_MAX} - \text{rand}()) / \text{static_cast}\langle\text{double}\rangle(\text{RAND_MAX})$
 - Type cast used to force double-precision division
- ❖ Random int between 1 & 6:
 $\text{rand}() \% 6 + 1$
 - "%" is modulus operator (remainder)
- ❖ Random int between 10 & 19:
 $\text{rand}() \% 10 + 10$



widelab
Web Information and Data Engineering Laboratory

Programmer-Defined Functions

- ❖ Write your own functions!
- ❖ Building blocks of programs
 - Divide & Conquer
 - Readability
 - Re-use
- ❖ Your "definition" can go in either:
 - Same file as main()
 - Separate file so others can use it, too

Components of Function Use

❖ 3 Pieces to using functions:

- Function Declaration/prototype
 - Information for compiler
 - To properly interpret calls
- Function Definition
 - Actual implementation/code for what function does
- Function Call
 - Transfer control to function

Function Declaration

- Also called function prototype
- An "informational" declaration for compiler
- Tells compiler how to interpret calls
 - Syntax:
`<return_type> FnName(<formal-parameter-list>);`
 - Example:
`double totalCost(int numberParameter,
double priceParameter);`
- Placed **before** any calls
 - In declaration space of main()
 - Or above main() in global space

Function Definition

- ❖ Implementation of function
- ❖ Just like implementing function main()

- ❖ Example:

```
double totalCost(int  
numberParameter,  
double priceParameter)  
{  
    const double TAXRATE = 0.05;  
    double subTotal;  
    subtotal = priceParameter * numberParameter;  
    return (subtotal + subtotal * TAXRATE);  
}
```

- ❖ Notice proper indenting

Function Definition Placement

- ❖ Placed after function main()
 - NOT "inside" function main()!
- ❖ Functions are "equals"; no function is ever "part" of another
- ❖ Formal parameters in definition
 - "Placeholders" for data sent in
 - "Variable name" used to refer to data in definition
- ❖ return statement
 - Sends data back to caller

Function Call

- ❖ Just like calling predefined function
`bill = totalCost(number, price);`
- ❖ Recall: `totalCost` returns double value
 - Assigned to variable named "bill"
- ❖ Arguments here: `number, price`
 - Recall arguments can be literals, variables, expressions, or combination
 - In function call, arguments often called "actual arguments"
 - Because they contain the "actual data" being sent

Function Example: Display 3.5 A Function of totalCost(1 of 2)

Display 3.5 A Function Using a Random Number Generator

```
1  #include <iostream>
2  using namespace std;

3  double totalCost(int numberParameter, double priceParameter);
4  //Computes the total cost, including 5% sales tax,
5  //on numberParameter items at a cost of priceParameter each.

6  int main( )
7  {
8      double price, bill;
9      int number;

10     cout << "Enter the number of items purchased: ";
11     cin >> number;
12     cout << "Enter the price per item $";
13     cin >> price;

14     bill = totalCost(number, price);
```

*Function declaration;
also called the function
prototype*

Function call

Function Example: Display 3.5 A Function of totalCost (2 of 2)

```
15     cout.setf(ios::fixed);  
16     cout.setf(ios::showpoint);  
17     cout.precision(2);  
18     cout << number << " items at "  
19         << "$" << price << " each.\n"  
20         << "Final bill, including tax, is $" << bill  
21         << endl;
```

```
22     return 0;  
23 }
```

```
24 double totalCost(int numberParameter, double priceParameter)  
25 {  
26     const double TAXRATE = 0.05; //5% sales tax  
27     double subtotal;  
  
28     subtotal = priceParameter * numberParameter;  
29     return (subtotal + subtotal*TAXRATE);  
30 }
```

Function
head

Function
body

Function
definition

SAMPLE DIALOGUE

Enter the number of items purchased: 2
Enter the price per item: \$10.10
2 items at \$10.10 each.
Final bill, including tax, is \$21.21



widelab
Web Information and Data Engineering Laboratory

Alternative Function Declaration

- ❖ Recall: Function declaration is "information " for compiler
- ❖ Compiler only needs to know:
 - Return type
 - Function name
 - Parameter list
- ❖ Formal parameter names not needed:
`double totalCost(int, double);`
 - Still "should" put in formal parameter names
 - Improves readability

Parameter vs. Argument

- ❖ Terms often used interchangeably
- ❖ Formal parameters/arguments
 - In function declaration
 - In function definition' s header
- ❖ Actual parameters/arguments
 - In function call
- ❖ Technically parameter is "formal" piece while argument is "actual" piece*
 - *Terms not always used this way



widelab
Web Information and Data Engineering Laboratory

Functions Calling Functions

- ❖ We' re already doing this!
 - `main()` IS a function!
- ❖ Only requirement:
 - Function' s declaration must appear first
- ❖ Function' s definition typically elsewhere
 - After `main()`'s definition
 - Or in separate file
- ❖ Common for functions to call many other functions
- ❖ Function can even call itself → "Recursion"

Self test

- ❖ 撰寫一個int InputInteger(),除了顯示一連串的提示語外,可以將不是正整數的數字轉成正整數
- ❖ 利用InputInteger()取得使用者輸入的數字,然後判斷這個數字是不是偶數,依據數字顯示" 此數字是偶數",或是" 此數字是奇數" (0視為偶數)



Boolean Return-Type Functions

❖ Return-type can be any valid type

- Given function declaration/prototype:
`bool appropriate(int rate);`
- And function's definition:

```
bool appropriate (int rate)
{
    return (((rate>=10)&&(rate<20))||(rate==0));
}
```
- Returns "true" or "false"
- Function call, from some other function:

```
if (appropriate(entered_rate))
    cout << "Rate is valid\n";
```

Declaring Void Functions

- ❖ Similar to functions returning a value
- ❖ Return type specified as "void"
- ❖ Example:
 - Function declaration/prototype:
`void showResults(double fDegrees,
double cDegrees);`
 - Return-type is "void"
 - Nothing is returned

Declaring Void Functions

❖ Function definition:

```
void showResults(double fDegrees, double  
cDegrees)  
{  
    cout.setf(ios::fixed);  
    cout.setf(ios::showpoint);  
    cout.precision(1);  
    cout << fDegrees  
        << " degrees fahrenheit equals \n"  
        << cDegrees << " degrees celsius.\n";  
}
```

❖ Notice: no return statement

- Optional for void functions



widelab
Web Information and Data Engineering Laboratory

Calling Void Functions

- ❖ Same as calling predefined void functions
- ❖ From some other function, like main():
 - `showResults(degreesF, degreesC);`
 - `showResults(32.5, 0.3);`
- ❖ Notice no assignment, since no value returned
- ❖ Actual arguments (degreesF, degreesC)
 - Passed to function
 - Function is called to "do it" s job" with the data passed in



More on Return Statements

- ❖ Transfers control back to "calling" function
 - For return type other than void, MUST have return statement
 - Typically the LAST statement in function definition
- ❖ return statement optional for void functions
 - Closing } would implicitly return control from void function

Preconditions and Postconditions

- ❖ Similar to "I-P-O" discussion
- ❖ Comment function declaration:

```
void showInterest(double balance, double rate);  
//Precondition: balance is nonnegative account balance  
//           rate is interest rate as percentage  
//Postcondition: amount of interest on given balance,  
//           at given rate ...
```
- ❖ Often called Inputs & Outputs

main(): "Special"

- ❖ Recall: main() IS a function
- ❖ "Special" in that:
 - One and only one function called main() will exist in a program
- ❖ Who calls main()?
 - Operating system
 - Tradition holds it should have return statement
 - Value returned to "caller" → Here: operating system
 - Should return "int" or "void"

Scope Rules

- ❖ Local variables
 - Declared inside body of given function
 - Available only within that function
- ❖ Can have variables with same names declared in different functions
 - Scope is local: "that function is it' s scope"
- ❖ Local variables preferred
 - Maintain individual control over data
 - Need to know basis
 - Functions should declare whatever local data needed to "do their job"



widelab
Web Information and Data Engineering Laboratory

Procedural Abstraction

- ❖ Need to know "what" function does, not "how" it does it!
- ❖ Think "black box"
 - Device you know how to use, but not it's method of operation
- ❖ Implement functions like black box
 - User of function only needs: declaration
 - Does NOT need function definition
 - Called Information Hiding
 - Hide details of "how" function does it's job

Self test

- ❖ 撰寫一個MAX function ,可以傳回三個整數型態變數的最大值
- ❖ 利用亂數產生器,產生3個介於1-100的整數,然後送入MAX function,找出這個三個數的最大值

Self test

❖ 撰寫一個STAR function, 輸入為 n , dir 兩個參數

■ 當 $dir = 1$ 時,可以依據傳回的變數 n ,呈現圖

```
*  
**  
***  
****  
*****
```

■ 當 $dir = 0$ 時,可以依據傳回的變數 n ,呈現圖

```
*****  
****  
***  
**  
*
```


Self test

- ❖ 設計一個Sum() function,當使用者輸入一個整數n,可以計算 $1+2+3+\dots+n$
- ❖ 設計一個Product() function 當使用者輸入一個整數n可以計算 $1*2*3*\dots*n$
- ❖ 利用switch case
 - 當使用者輸入" S" 或是" s" ,執行Sum()
 - 當使用者輸入" P" 或是" p" ,執行Product()
 - 當使用者輸入" E" 或是" e" ,結束程式
 - 按其他按鍵重新執行一次

如何判斷輸入的是數字？

```
#include <iostream>
using namespace std;

bool isDigit(char c) {
    return (c >= '0' && c <= '9'); // 檢查字符是否在 '0' 到 '9' 之間
}

int main() {
    char input;
    cout << "請輸入一個字元：";
    cin >> input;

    if (isDigit(input)) {
        cout << "輸入的是數字。" << endl;
    } else {
        cout << "輸入的不是數字。" << endl;
    }

    return 0;
}
```

如何判斷輸入的是字母？

```
#include <iostream>
using namespace std;
```

```
bool isAlphabet(char c) {
    return (c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z'); // 檢查字符是否在 'A'-'Z'
    或 'a'-'z' 之間
}
```

```
int main() {
    char input;
    cout << "請輸入一個字元 : ";
    cin >> input;

    if (isAlphabet(input)) {
        cout << "輸入的是英文字母。" << endl;
    } else {
        cout << "輸入的不是英文字母。" << endl;
    }
}
```

```
return 0;
```

```
}
```

一個數是質數嗎？

```
bool isPrime(int n)
{
    if (n <= 1) return false; // 1 或更小的數不是質數
    if (n == 2 || n == 3) return true; // 2 和 3 是質數
    for (int i = 2; i < n; i++)
    {
        if (n % i == 0) return false;
    }
    return true; // 如果沒有找到因數，則是質數
}
```


一個數是質數嗎？

```
bool isPrime(int n)
{
    if (n <= 1) return false; // 1 或更小的數不是質數
    if (n <= 3) return true; // 2 和 3 是質數
    if (n % 2 == 0 || n % 3 == 0) return false; // 排除 2 和 3 的倍數

    int limit = sqrt(n); // 只檢查到平方根即可
    for (int i = 5; i <= limit; i += 6) { // 只檢查形如  $6k \pm 1$  的數
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }

    return true;
}
```

Self test

❖ 利用亂數產生器產生三個數字1-100整數

- 撰寫一個int even() function,計算有幾個偶數
- 撰寫一個int odd() function,計算有幾個奇數

Global Constants and Global Variables

- ❖ Declared "outside" function body
 - Global to all functions in that file
- ❖ Declared "inside" function body
 - Local to that function
- ❖ Global declarations typical for constants:
 - `const double TAXRATE = 0.05;`
 - Declare globally so all functions have scope
- ❖ Global variables?
 - Possible, but SELDOM-USED
 - Dangerous: no control over usage!

Global variable

```
void Fun_A();  
int a =10;  
  
int main()  
{  
    cout<<a<<endl;  
    Fun_A();  
    cout<<a<<endl;  
}  
void Fun_A()  
{  
    a++;  
}
```


Blocks

- ❖ Declare data inside compound statement
 - Called a "block"
 - Has "block-scope"
- ❖ Note: all function definitions are blocks!
 - This provides local "function-scope"
- ❖ Loop blocks:

```
for (int ctr=0;ctr<10;ctr++)  
{  
    sum+=ctr;  
}
```

 - Variable ctr has scope in loop body block only

Nested Scope

- ❖ Same name variables declared in multiple blocks
- ❖ Very legal; scope is "block-scope"
 - No ambiguity
 - Each name is distinct within its scope

Summary 1

- ❖ Two kinds of functions:
 - "Return-a-value" and void functions
- ❖ Functions should be "black boxes"
 - Hide "how" details
 - Declare own local data
- ❖ Function declarations should self-document
 - Provide pre- & post-conditions in comments
 - Provide all "caller" needs for use

Summary 2

- ❖ Local data
 - Declared in function definition
- ❖ Global data
 - Declared above function definitions
 - OK for constants, not for variables
- ❖ Parameters/Arguments
 - Formal: In function declaration and definition
 - Placeholder for incoming data
 - Actual: In function call
 - Actual data passed to function

Parameters

Learning Objectives

❖ Parameters

- Call-by-value
- Call-by-reference
- Mixed parameter-lists

❖ Overloading and Default Arguments

- Examples, Rules

Parameters

- ❖ Two methods of passing arguments as parameters
- ❖ Call-by-value
 - "copy" of value is passed
- ❖ Call-by-reference
 - "address of" actual argument is passed

Call-by-Value Parameters

- ❖ Copy of actual argument passed
- ❖ Considered "local variable" inside function
- ❖ If modified, only "local copy" changes
 - Function has no access to "actual argument" from caller
- ❖ This is the default method
 - Used in all examples thus far

Call-by-Value Example: Display 4.1 Formal Parameter Used as a Local Variable (1 of 3)

Display 4.1 Formal Parameter Used as a Local Variable

```
1 //Law office billing program.
2 #include <iostream>
3 using namespace std;

4 const double RATE = 150.00; //Dollars per quarter hour.

5 double fee(int hoursWorked, int minutesWorked);
6 //Returns the charges for hoursWorked hours and
7 //minutesWorked minutes of legal services.

8 int main()
9 {
10     int hours, minutes;
11     double bill;
```

Call-by-Value Example: Display 4.1 Formal Parameter Used as a Local Variable (2 of 3)

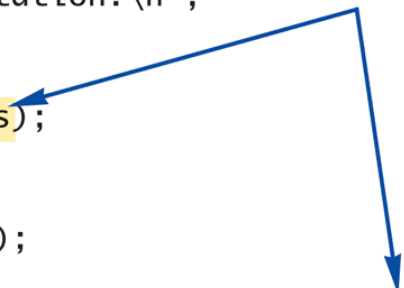
```
12      cout << "Welcome to the law office of\n"
13          << "Dewey, Cheatham, and Howe.\n"
14          << "The law office with a heart.\n"
15          << "Enter the hours and minutes"
16          << " of your consultation:\n";
17      cin >> hours >> minutes;

18      bill = fee(hours, minutes);

19      cout.setf(ios::fixed);
20      cout.setf(ios::showpoint);
21      cout.precision(2);
22      cout << "For " << hours << " hours and " << minutes
23          << " minutes, your bill is $" << bill << endl;

24      return 0;
25  }
```

*The value of minutes
is not changed by the
call to fee.*



(continued)

Call-by-Value Example: Display 4.1 Formal Parameter Used as a Local Variable (3 of 3)

Display 4.1 Formal Parameter Used as a Local Variable

```
26 double fee(int hoursWorked, int minutesWorked)
27 {
28     int quarterHours;

29     minutesWorked = hoursWorked*60 + minutesWorked;
30     quarterHours = minutesWorked/15;
31     return (quarterHours*RATE);
32 }
```

*minutesWorked is a local
variable initialized to the
value of minutes.*

SAMPLE DIALOGUE

Welcome to the law office of
Dewey, Cheatham, and Howe.
The law office with a heart.

Enter the hours and minutes of your consultation:

5 46

For 5 hours and 46 minutes, your bill is \$3450.00

Call-by-Value Pitfall

❖ Common Mistake:

- Declaring parameter "again" inside function:
double fee(int hoursWorked, int
minutesWorked)

```
{  
    int quarterHours;           // local variable  
    int minutesWorked          // NO!  
}
```

- Compiler error results
 - "Redefinition error..."

❖ Value arguments ARE like "local variables"

- But function gets them "automatically"

Call-By-Reference Parameters

- ❖ Used to provide access to caller's actual argument
- ❖ Caller's data can be modified by called function!
- ❖ Typically used for input function
 - To retrieve data for caller
 - Data is then "given" to caller
- ❖ Specified by ampersand, &, after type in formal parameter list

Call-By-Reference Example: Display 4.1 Call-by-Reference Parameters (1 of 3)

Display 4.2 Call-by-Reference Parameters

```
1  //Program to demonstrate call-by-reference parameters.
2  #include <iostream>
3  using namespace std;

4  void getNumbers(int& input1, int& input2);
5  //Reads two integers from the keyboard.

6  void swapValues(int& variable1, int& variable2);
7  //Interchanges the values of variable1 and variable2.

8  void showResults(int output1, int output2);
9  //Shows the values of variable1 and variable2, in that order.

10 int main()
11 {
12     int firstNum, secondNum;

13     getNumbers(firstNum, secondNum);
14     swapValues(firstNum, secondNum);
15     showResults(firstNum, secondNum);
16     return 0;
17 }
```

Call-By-Reference Example: Display 4.1 Call-by-Reference Parameters (2 of 3)

```
18 void getNumbers(int& input1, int& input2)
19 {
20     cout << "Enter two integers: ";
21     cin >> input1
22     >> input2;
23 }

24 void swapValues(int& variable1, int& variable2)
25 {
26     int temp;

27     temp = variable1;
28     variable1 = variable2;
29     variable2 = temp;
30 }
31
32 void showResults(int output1, int output2)
33 {
34     cout << "In reverse order the numbers are: "
35     << output1 << " " << output2 << endl;
36 }
```

Call-By-Reference Example: Display 4.1 Call-by-Reference Parameters (3 of 3)

Display 4.2 Call-by-Reference Parameters

SAMPLE DIALOGUE

Enter two integers: 5 6

In reverse order the numbers are: 6 5

Call-By-Reference Details

- ❖ What's really passed in?
- ❖ A "reference" back to caller's actual argument!
 - Refers to memory location of actual argument
 - Called "address", which is a unique number referring to distinct place in memory



Constant Reference Parameters

- ❖ Reference arguments inherently "dangerous"
 - Caller's data can be changed
 - Often this is desired, sometimes not
- ❖ To "protect" data, & still pass by reference:
 - Use const keyword
 - `void sendConstRef(const int &par1,
const int &par2);`
 - Makes arguments "read-only" by function
 - No changes allowed inside function body

Parameters and Arguments

- ❖ Confusing terms, often used interchangeably
- ❖ True meanings:
 - Formal parameters
 - In function declaration and function definition
 - Arguments
 - Used to "fill-in" a formal parameter
 - In function call (argument list)
 - Call-by-value & Call-by-reference
 - Simply the "mechanism" used in plug-in process

Mixed Parameter Lists

- ❖ Can combine passing mechanisms
- ❖ Parameter lists can include pass-by-value and pass-by-reference parameters
- ❖ Order of arguments in list is critical:
void mixedCall(int & par1, int par2, double & par3);
 - Function call:
mixedCall(arg1, arg2, arg3);
 - arg1 must be integer type, is passed by reference
 - arg2 must be integer type, is passed by value
 - arg3 must be double type, is passed by reference

Self-test exercise

```
VoidMyFun(i nt& a, int b, int& c);  
int main()  
{  
    int x=10,y=20,z=30;  
    cout <<x<< " , " <<y<< " , " <<z<<endl;  
    MyFun(x,y,z);  
    cout <<x<< " , " <<y<< " , " <<z<<endl;  
}  
VoidMyFun(i nt& a, int b, int& c)  
{  
    a=1;  
    b=2;  
    c=3;  
}
```

Overloading

- ❖ Same function name
- ❖ Different parameter lists
- ❖ Two separate function definitions
- ❖ Function "signature"
 - Function name & parameter list
 - Must be "unique" for each function definition
- ❖ Allows same task performed on different data



Overloading Example: Average

- ❖ Function computes average of 2 numbers:

```
double average(double n1, double n2)
{
    return ((n1 + n2) / 2.0);
}
```
- ❖ Now compute average of 3 numbers:

```
double average(double n1, double n2, double n3)
{
    return ((n1 + n2+n3) / 3.0);
}
```
- ❖ Same name, two functions

Overloaded Average() Cont' d

- ❖ Which function gets called?
- ❖ Depends on function call itself:
 - `avg = average(5.2, 6.7);`
 - Calls "two-parameter average()"
 - `avg = average(6.5, 8.5, 4.2);`
 - Calls "three-parameter average()"
- ❖ Compiler resolves invocation based on signature of function call
 - "Matches" call with appropriate function
 - Each considered separate function

Self test

❖ Given following functions:

- #1. void f(double n, int m);
- #2. void f(int n, int m);
- These calls:
 - f(98, 99); → Calls ??
 - f(5.3, 4); → Calls ?
 - f(4.3, 5.2); → Calls ???

Overloading Pitfall

- ❖ Only overload "same-task" functions
 - A mpg() function should always perform same task, in all overloads
 - Otherwise, unpredictable results
- ❖ C++ function call resolution:
 - 1st: looks for exact signature
 - 2nd: looks for "compatible" signature

Overloading Resolution

❖ 1st: Exact Match

- Looks for exact signature
 - Where no argument conversion required

❖ 2nd: Compatible Match

- Looks for "compatible" signature where automatic type conversion is possible:
 - 1st with promotion (e.g., int→double)
 - No loss of data
 - 2nd with demotion (e.g., double→int)
 - Possible loss of data

Automatic Type Conversion and Overloading

- ❖ Numeric formal parameters typically made "double" type
- ❖ Allows for "any" numeric type
 - Any "subordinate" data automatically promoted
 - int → double
 - float → double
 - char → double
- ❖ Avoids overloading for different numeric types

*More on this later!

Automatic Type Conversion and Overloading Example

❖ `double mpg(double miles, double gallons)`
{
 return (miles/gallons);
}

❖ Example function calls:

- `mpgComputed = mpg(5, 20);`
 - Converts 5 & 20 to doubles, then passes
- `mpgComputed = mpg(5.8, 20.2);`
 - No conversion necessary
- `mpgComputed = mpg(5, 2.4);`
 - Converts 5 to 5.0, then passes values to function

Default Arguments

- ❖ Allows omitting some arguments
- ❖ Specified in function declaration/prototype
 - `void showVolume(int length,
 int width = 1,
 int height = 1);`
 - Last 2 arguments are defaulted
 - Possible calls:
 - `showVolume(2, 4, 6);` //All arguments supplied
 - `showVolume(3, 5);` //height defaulted to 1
 - `showVolume(7);` //width & height defaulted to 1

Self-test Exercise

Display 4.8 Default Arguments

```

1  #include <iostream>
2  using namespace std;

3  void showVolume(int length, int width = 1, int height = 1);
4  //Returns the volume of a box.
5  //If no height is given, the height is assumed to be 1.
6  //If neither height nor width is given, both are assumed to be 1.

7  int main( )
8  {
9      showVolume(4, 6, 2);
10     showVolume(4, 6);
11     showVolume(4);

12     return 0;
13 }

14 void showVolume(int length, int width, int height)

```

Default arguments

A default argument should not be given a second time.

```

16 {
17     cout << "Volume of a box with \n"
18         << "Length = " << length << ", Width = " << width << endl
19         << "and Height = " << height
20         << " is " << length*width*height << endl;
21 }

```

Answer

```
16 {  
17     cout << "Volume of a box with \n"  
18         << "Length = " << length << ", Width = " << width << endl  
19         << "and Height = " << height  
20         << " is " << length*width*height << endl;  
21 }
```

SAMPLE DIALOGUE

Volume of a box with
Length = 4, Width = 6
and Height = 2 is 48
Volume of a box with
Length = 4, Width = 6
and Height = 1 is 24
Volume of a box with
Length = 4, Width = 1
and Height = 1 is 4